DEPLOYADA

RELEASE 1 · SPRINT 04

Docker

o fim do "mas na minha máquina funciona" ♥



material de apoio da sprint

@deployada

O delivery do Docker

O restaurante começou a entregar. E o prato tem que chegar **igualzinho** em qualquer casa.

A marmita lacrada

container

O código e tudo que ele precisa, rodando igual em qualquer lugar.

A ficha técnica

Dockerfile

O passo a passo pra montar a marmita.

A marmita pronta e congelada

imagem

Montada a partir da ficha. Não muda mais.

O mercado de marmitas

Docker Hub

Imagens prontas: Postgres, MySQL, Redis, nginx...

O pedido completo

Docker Compose

Prato, bebida e sobremesa: vários containers juntos.

A geladeira fora da marmita

volume

Onde os dados ficam guardados, mesmo se a marmita for pro lixo.

Da ficha até a mesa

Três palavras que confundem muita gente, na ordem em que acontecem.



Dockerfile

a receita

`docker build` →



imagem

a marmita pronta

`docker run` →



container

a marmita servida

Lotou o restaurante? **De uma imagem só** você sobe quantos containers quiser, todos iguaizinhos.



e a máquina virtual?

Máquina virtual

Manda a cozinha inteira pra casa do cliente: fogão, geladeira e pia. Funciona, mas é pesado.

Container

Manda só o prato e o que ele precisa. Usa a "cozinha" da máquina, por isso é leve e sobe em segundos.

Colinha de comandos

~/restaurante — bash

`docker build -t nome .`	Monta a imagem a partir do Dockerfile da pasta.
`docker run imagem`	Cria e liga um container a partir da imagem.
`docker run -d -p 8080:80 nginx`	Roda em segundo plano e liga a porta 8080 da sua máquina à 80 do container.
`docker ps`	Mostra os containers rodando. `docker ps -a mostra todos, até os parados`
`docker logs id`	Mostra o que o container escreveu. Ótimo pra achar erro.
`docker stop id`	Desliga o container.
`docker rm id`	Apaga o container (a imagem continua).
`docker compose up -d`	Sobe todos os containers do compose.yaml.
`docker compose down`	Derruba tudo. Os volumes ficam. `com -v, apaga os volumes também (cuidado!)`

Uma ficha técnica por dentro

Um Dockerfile de uma API em Java. **Cada linha é um passo da receita.**

`FROM eclipse-temurin:21-jre`	Começa com uma base que já tem o Java 21.
`WORKDIR /app`	Escolhe a "bancada" de trabalho dentro da marmita.
`COPY app.jar .`	Coloca o seu código pronto lá dentro.
`EXPOSE 8080`	Avisa em qual porta a aplicação atende.
`CMD ["java","-jar","app.jar"]`	O que rodar quando a marmita for aberta.

1. Comece de uma **imagem oficial** do Docker Hub. Não reinvente o fogão.
2. Fixe a versão: **21-jre**, e não "latest". Assim a receita não muda sozinha.
3. Nada de senha no Dockerfile. Ela fica gravada na imagem.

Deu ruim em produção

🔥 O CASO

Um banco Postgres rodando num container, cheio de dados. Na hora de atualizar a versão, o container antigo foi apagado e um novo subiu. **O banco voltou vazio.**

os dados estavam dentro da marmita!

1 **Container é descartável**
Apagou o container, foi junto tudo que estava só dentro dele.

2 **Dado mora no volume**
A geladeira fora da marmita. O container vai e volta, o volume fica.

3 **Volume não é backup**
Dado importante tem backup também.

```
# compose.yaml
services:
  db:
    image: postgres:16
    volumes:
      - dados-db:/var/lib/postgresql/data
volumes:
  dados-db:
```

Missão da sprint

Rode os seus **primeiros containers**. Leva uns 15 minutos.

1. Instale o Docker Desktop. É gratuito pra uso pessoal.

2. No terminal, rode `docker run hello-world`. Apareceu a mensagem de boas-vindas? Funcionou!

3. Agora rode `docker run -d -p 8080:80 nginx`.

4. Abra `localhost:8080` no navegador. "Welcome to nginx!" está saindo de dentro de uma marmita.

5. Veja o container com `docker ps` e desligue com `docker stop` + o ID.

Manda o print da página do nginx no **@deployada**!

Release 2

até o próximo deploy!

FIM DA RELEASE 1 ✓

No próximo episódio: Retrospectiva da Release

e depois a Release 2 começa com Kubernetes

Ouça no Spotify, no Deezer ou direto pelo site.
Tudo no link da bio do **@deployada**

Release 1